Flink as an Isotope Collector

Two propagation models now coexist in this project. They emit the identical wire format, so `05_isotope_view.fql` and every report is unchanged and cannot tell the collectors apart.

	Out-of-band propagation	In-band propagation
Where	Services in `app/`	Flink SQL (CP + CCAF)
Carrier	`IsotopeContext ThreadLocal`	The record's own Kafka headers
Stamped by	`IsotopeProducerInterceptor.onSend()`	`ISOTOPE_APPEND_HOP(...)` in `INSERT INTO`
Activation	`interceptor.classes` config	`CREATE FUNCTION` + a writable `headers` column

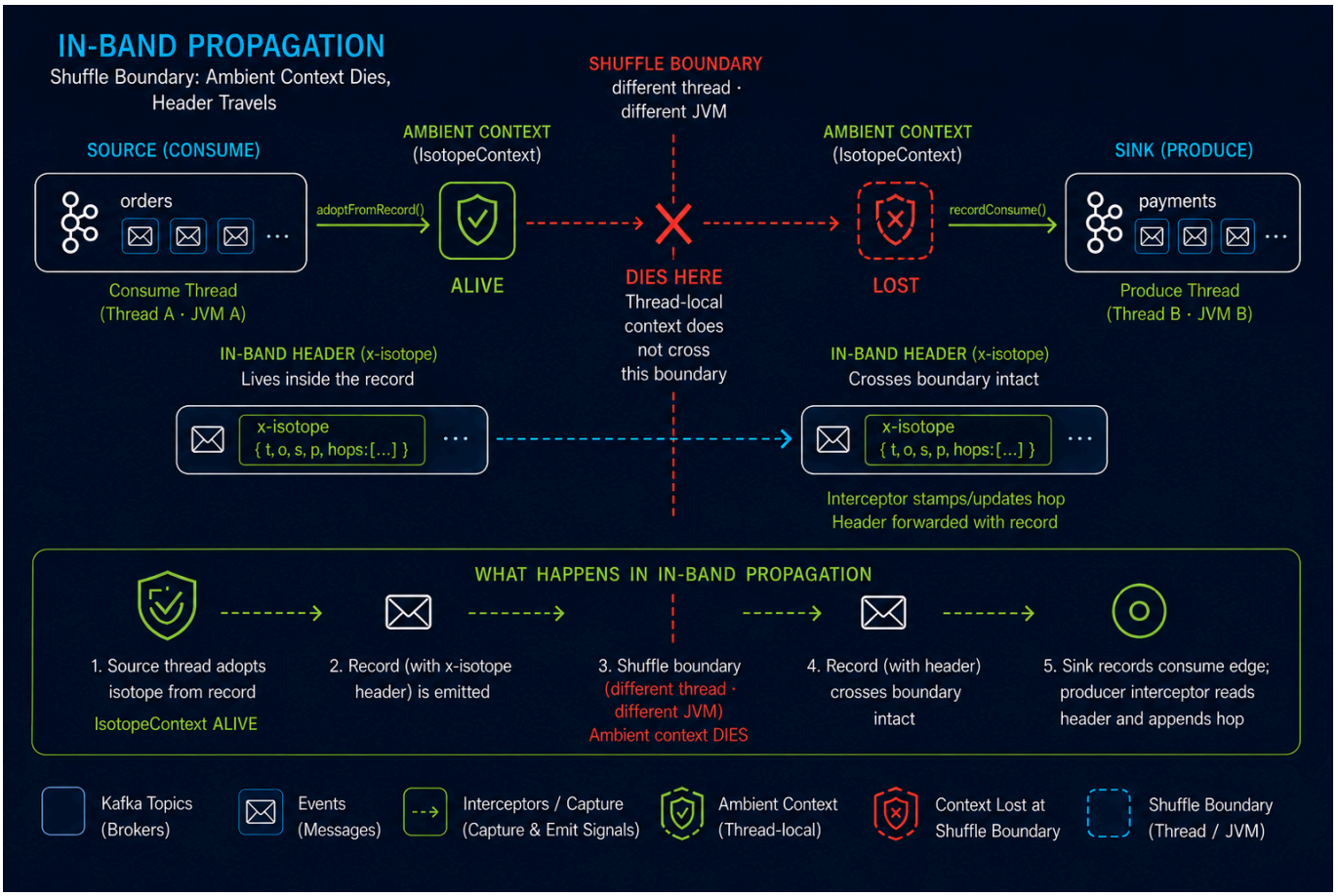
Table of Contents

- 1.0 Why a second model was needed
- 2.0 What is wired, and where
  - 2.1 The writable headers column
  - 2.2 CCAF table shape
  - 2.3 Packaging the UDF
  - 2.4 [OPTIONAL] Fan-in provenance
- 3.0 Constraints
  - 3.1 1:1 Statements Only
  - 3.2 Header Keys Must Be Unique
  - 3.3 Windowed Fan-in Only
  - 3.4 Append-only Sources Only
- 4.0 What neither addition changes

1.0 Why a second model was needed

Out-of-band propagation is correct where it runs: one thread owns a whole consume→produce hop, so a `ThreadLocal` set at consume time is still there at `send()` time.

It cannot work in Flink. A shuffle serializes a record and resumes it on a different thread, usually in a different TaskManager JVM, so anything set *beside* the record is gone. Flink SQL is stricter still — there is no user-visible thread to attach to and no way to carry an opaque value past the planner, which is free to reorder, fuse, and eliminate rows.



The shuffle boundary is the whole argument. **Out-of-band**, the isotope lives *beside* the record and dies there; **in-band**, it lives *inside* the record and crosses intact.

> In Apache Flink, the term **shuffle (or data shuffling)** refers to the *process of redistributing data across different parallel partitions or network channels*. It is a critical operation used when data needs to be regrouped, combined, or re-organized across a distributed cluster—such as when performing a key-by grouping, a windowed aggregation, or a distributed join.

So in-band propagation moves the isotope *into* the record — which does not make the planner gentler, it makes the isotope something the planner is **obligated to respect**. `ISOTOPE_APPEND_HOP(...)` is an ordinary scalar expression over column values, and the planner may still reorder, fuse, push down, or split that plan across a shuffle. What it may not do is change the values the query computes. Carrying a column value is precisely the thing no rewrite is allowed to optimize away, so the isotope arrives at the sink intact.

The `ThreadLocal` had no such protection, for a precise reason: no expression in the query referenced it, so the planner saw no dependency and owed it nothing. The fix is not to work around the optimizer but to state the requirement in terms it already guarantees.

Two consequences follow, both intentional. The expression references `headers`, so projection pushdown must now keep that column alive from source to sink — a real dependency, visible to the optimizer. And Flink assumes a `ScalarFunction` is deterministic unless told otherwise, meaning it may constant-fold one, eliminate it as a common subexpression, or recompute it on failover; that is why the hop timestamp is a parameter rather than a clock read inside `eval()`. See §3.0 Constraints.

Note this is not a Flink limitation being worked around. Nothing in Flink core needs to change; the out-of-band *delivery mechanism* is what doesn't port, while the isotope format and semantics port intact.

2.0 What is wired, and where

Flink collects onto a **new parallel topic**, `orders.flink_enriched`. It reads `orders.placed` and re-emits each record with one extra hop. The three-stage service pipeline is untouched — `orders.{placed,enriched,fulfilled}` still belong entirely to out-of-band propagation — and Flink now appears as a producer in `topology_1m` and `bipartite_topology_1m`, where it was previously invisible.

Piece	CP (minikube)	CCAF
UDF registration	`01_register_functions.fql`	`register_isotope_append_hop` in `setup-ccaf.tf`
Writable sink table	`07_flink_collector_sink.fql`	`flink_collector_sink` + two `ALTER`s in `setup-ccaf.tf`
Collector INSERT	`75_flink_collector.fql`	`insert_flink_collector` in `setup-ccaf.tf`
Teardown	`99_teardown.fql`	`terraform destroy`

`IsotopeReportsJob` registers `ISOTOPE_APPEND_HOP` from the classpath and runs the collector INSERT alongside the seven report INSERTs in the same `StatementSet`.

2.1 The writable headers column

The sink declares `headers` **without** `VIRTUAL`, which is what makes it writable — virtual metadata columns are read-only and excluded from the `INSERT INTO` target schema. On CCAF the equivalent is `ALTER TABLE ... MODIFY \`headers` MAP<STRING, STRING> METADATA, since the four existing` ALTERs add the column as METADATA VIRTUAL`.

> Once persisted, the column is **mandatory in every `INSERT INTO`** that table. Only make it writable on tables Flink actually writes.

The source tables keep `MAP<STRING, BYTES>` (the Kafka connector's native metadata type) and the call site casts to `MAP<STRING, STRING>` on the way in. That keeps one UDF signature across both runtimes instead of forking the Java, and `MAP<STRING, STRING>` is the type Confluent documents for reading *and* writing headers.

The hop timestamp is passed **in** as `UNIX_TIMESTAMP() * 1000` rather than read inside `eval()` — a built-in the planner already knows not to fold, which keeps the non-determinism somewhere the optimizer can see it rather than hidden in the Java. See §1.0 Why a second model was needed.

2.2 CCAF table shape

`DESCRIBE \`orders.placed`` against the live cluster returns:

Column	Type	Extras
`key`	`BYTES`	`BUCKET KEY`
`val`	`BYTES`	

Column	Type	Extras
headers	MAP<STRING, BYTES>	METADATA FROM 'headers' VIRTUAL

The Topic Catalog auto-imported it as the bytes-pair table rather than deriving columns from the Protobuf schema, so the CCAF sink mirrors `(key, val)` while the CP side declares a single `value` BYTES column locally. Each runtime keeps the shape its own catalog produces; only the header handling is shared.

2.3 Packaging the UDF

The UDF ships as the `ptf/` shadow JAR. One packaging detail is load-bearing, and getting it wrong fails on the first row rather than at registration.

`Isotope` declares `fromHeaders(org.apache.kafka.common.header.Headers)`. `IsotopeAppendHop` never calls it, but the JVM resolves that descriptor when linking the class, so `Headers` must be present at **runtime**, not merely at compile time.

`compileOnly` was tried first, on the assumption that the Flink Kafka connector would supply `kafka-clients`. It does not on CCAF. Probed against the live compute pool, `CREATE FUNCTION` succeeded and the statement planned and reached `RUNNING` — then died on the first row:

```
phase:  FAILED
detail: UDF invocation error: exception raised in the user function code
        Message: org.apache.kafka.common.header.Headers
```

That failure timing is the trap: CCAF's function classloader does not expose `kafka-clients`, and nothing at registration time says so.

So `ptf/build.gradle` bundles `kafka-clients` (`transitive = false`, so no compression codecs) and relocates `org.apache.kafka` → `ai.signalroom.shaded.kafka`, which rewrites `Isotope`'s own reference in step and keeps the bundled copy from ever meeting the runtime's. It then ships **only** `common/header`, dropping the rest by package: 2 classes and ~1 KB, against 4,152 classes and 12 MB for the whole client.

`minimize()` cannot do that job — a type named only in a method descriptor is invisible to bytecode reachability analysis, so minimize strips `Headers` and the artifact fails exactly as `compileOnly` did. Only `Isotope` is ever loaded by the UDF; `IsotopeContext`, whose descriptors do reach `ConsumerRecord` and `Producer`, is never touched, and class loading is lazy, so its missing types cost nothing.

Confirmed in production on CCAF. With the relocated build deployed, querying the collector's output returns the stamped headers:

```
SELECT `headers`['x-isotope-this-service'], `headers`['x-isotope-hop-
count']
FROM `orders.flink_enriched` LIMIT 3;
```

```
flink-enrich    2
flink-enrich    2
flink-enrich    2
```

`hop_count = 2` is the load-bearing part: hop 1 was the origin service producing to `orders.placed`, hop 2 is Flink. The count incremented rather than resetting, so the UDF rehydrated the inbound isotope and appended to it rather than minting a fresh trace — the same assertion `IsotopeAppendHopTest` makes, now observed end to end.

The durable fix remains a Kafka-free `isotope-format` artifact split out of `kafka-isotope`, which would make all of this unnecessary.

2.4 [OPTIONAL] Fan-in provenance

Everything above is the 1:1 collector, which is always on. A second, **opt-in** collector handles the case §3.1 rules out for in-band propagation: a windowed aggregate whose output is a business event rather than a terminal report.

It writes two topics:

Topic	Carries
`orders.flink_batched`	The merged event — a 1-minute batch summary per pipeline, carrying a **fresh** isotope trace with one hop.
`isotope_merge_edge_markers`	The many-to-one edges: `(merge_trace_id, window_start, window_end, operator, pipeline, contributing_trace_id, contributing_service, contributing_topic, contributing_records)`, one row per contributing **trace** per window, with `contributing_records` carrying how many records that parent fed.

Join the two on `merge_trace_id` to recover any merged record's full parent set. The merged record does **not** continue a parent's trace, because a `SUM` over 1,000 records has 1,000 parents and naming one of them would fabricate provenance rather than report it.

The two statements — `80_merge_collector.fql` and `81_merge_edge_markers.fql` — read the same window, one aggregating it and one projecting every row in it. Nothing holds them together except that both derive the *same* `merge_trace_id` from `(pipeline, operator, window_start, window_end, group_key)` via `ISOTOPE_MERGE_TRACE` and `ISOTOPE_MERGE_TRACE_ID`. **Their argument lists must stay identical**, including the `WHERE` clause that selects input rows; `MergeTraceTest` pins the Java half of that contract, but the two SQL statements are edited by hand.

Turning it on:

```
# CP — minikube
make cp-flink-reports-up ENABLE_MERGE_PROVENANCE=true
```

```
# CCAF
make cc-flink-reports-up ENABLE_MERGE_PROVENANCE=true \
    CONFLUENT_API_KEY=$CONFLUENT_API_KEY
CONFLUENT_API_SECRET=$CONFLUENT_API_SECRET
```

Same switch name on both runtimes, and the same shape as `ENABLE_TRACE_RCA`. It reaches CP as the `--merge-provenance` job argument and CCAF as `TF_VAR_enable_merge_provenance`, so the underlying `MERGE_PROVENANCE=true scripts/deploy-cmf-flink-reports.sh up` and `terraform apply -var enable_merge_provenance=true` forms still work if you drive either directly.

Off, neither runtime creates a table, a topic, or a statement for it. The two UDFs are registered either way — registration is inert until a statement calls it, and always-registering them means flipping the feature on needs no artifact re-upload.

Both statements are window aggregates, and that is deliberate: a windowing TVF used as a bare projection has no watermark gate at all, so late records would emit edges for a merged record already published without them, and the edge list would drift above the parent set permanently. Grouping puts both behind the same window operator, so a late record is dropped by both. It also deduplicates — an edge list is a set of edges, and the same parent trace contributing twice is one derivation edge, so the *weight* of that edge lives in a `contributing_records` column rather than in duplicate rows. Nothing is lost, and both reconciliations become exact and worth asserting in tests: **the edge count for a `merge_trace_id` equals that merged record's `distinct_traces`, and `SUM(contributing_records)` equals its `event_count`.**

Two costs, stated plainly. The edge stream writes one record per contributing trace per window, a material fraction of that stage's write volume — inherent to materializing an edge list. And `orders.flink_batched` is deliberately **not** added to the `isotope_raw` union, exactly as `orders.flink_enriched` is not: the union is typed `MAP<STRING, BYTES>` and the collector sinks are `MAP<STRING, STRING>`, and keeping them out is what lets this feature be optional without the seven reports changing shape.

Which merged records did a stuck trace feed? The edges answer this, because `contributing_trace_id` and a stuck-trace alert's `trace_id` are the same identity. `82_merge_impact_query.fql` holds two read-only queries: one lists the merged records each stuck trace fed and where the trace stopped, and the other reports how many of each merged record's traces have alerted. Neither is deployed or writes to a topic. A merge reads the origin stream, so it counts a trace as soon as it is placed. Impact therefore means a merged record counted a trace that did not finish, not that its figures are wrong.

3.0 Constraints

3.1 1:1 Statements Only

1:1 statements only — tracing, not provenance. This is a deliberate boundary, not an unfinished feature, and the distinction is worth stating exactly.

An isotope records an *itinerary*: one identity and the ordered sequence of hops it visited — **a path**. Provenance records a *derivation*: for each output, the inputs that produced it — **a DAG**, and inherently many-to-one wherever data combines. The two coincide as long as every step is 1:1 — a DAG in which every node has exactly one parent is still a path — which is why `IsotopeAppendHop` and `IsotopeProducerInterceptor` both work.

A windowed aggregate breaks that 1:1 relationship. A `SUM` over 1,000 records has **1,000 parents**, and truthful provenance would need to identify all of them; the isotope format has no vocabulary for that. Therefore, **in-band propagation is restricted to 1:1 statements**, where a trace *is* a valid derivation record. Aggregations never emit a "representative" trace ID, because doing so would not merely produce incomplete provenance — **it would fabricate provenance by falsely implying that one input record produced the aggregate result.**

Note this is a property of the model, not of Flink: Kafka Streams and Spark face it identically. And in this pipeline it costs nothing, because the fan-in statements are all reports, and reports are terminal *as traced records*. `isotope_report_*_1m` is read by dashboards, never re-consumed as a business event that must carry a hop chain onward.

The trace-RCA report is the one case that reads a report rather than the event stream — `setup-ccaf-ai.tf` joins `isotope_report_stuck_trace_1m` to `ML_PREDICT` and emits an AI-written root cause. It is not a counterexample on either count: it is 1:1 (one alert in, one analysis out, via `LATERAL TABLE`), and it carries `trace_id` forward as a *column*, not as an isotope header, so it never re-enters the hop chain. Note it is CCAF-only and gated on `var.enable_trace_rca`; CP runs seven reports, CCAF seven or eight.

The 1:1 boundary happens to land exactly where tracing needs to reach.

Full provenance would **not** require a change to the isotope format. The fix is out-of-band rather than in-band: the merged output carries a **fresh trace**, while a side-channel *merge-edge* topic records the many-to-one edges — `(merge_trace_id, window_start, window_end, operator, contributing_trace_id, contributing_records)`, one row per contributing trace per window — following the same architectural pattern `isotope_consume_edge_markers` already uses for consume edges. The wire format remains untouched, and `isotope_raw`, the typed views, and all seven reports remain exactly as they are.

Carrying that provenance **in-band** is what is impractical. An aggregation over 1,000 inputs would require 1,000 UUIDv7 trace IDs — **16 KB of raw IDs alone** against Kafka's typical 1 MB `max.message.bytes`, before encoding, hop history, headers, or the business payload. `Isotope` already acknowledges this boundedness through `MAX_HOPS` and the `truncated` flag: the format is intentionally not designed to carry unbounded provenance history along a path. A fresh trace at the merge is therefore not a workaround for that limitation — **it is the correct identity for the new record created by the merge**.

The load-bearing problem is **determinism, not vocabulary**. `Isotope.newTrace` currently mints its UUIDv7 using `ThreadLocalRandom`, which is harmless because the mint path in `IsotopeAppendHop` only fires for records that arrive untraced. At a merge, however, minting becomes the normal path. Because the merged record and its provenance edges are emitted by two separate statements over the same window, independently generating their IDs would produce different `merge_trace_id` values under replay or re-evaluation, silently severing the relationship between them.

The merge ID therefore must be **derived rather than drawn**: a UUIDv7 whose timestamp field is the window end and whose random field is a truncated hash of `(pipeline, operator, window_start, window_end, group_key)`. Both statements can then independently compute the same ID; replay produces the same ID; and UUID ordering still follows event time. This extends the argument already made in §2.1 for passing the hop timestamp into `eval()` rather than reading the clock inside it: **anything that becomes part of trace identity must come from deterministic event data, not runtime state.**

That derivation needs a way to supply the trace ID, and `Isotope` does not offer one: its all-args constructor is package-private and its only public factory, `newTrace(service, pipeline, originTsMs)`, always mints its own random ID. Rather than fork the format knowledge that the shared `kafka-isotope-core` dependency exists to centralize, `MergeTrace` builds a real `Isotope`, re-serializes it, and substitutes the single Base64 trace-ID field before parsing it back through the public `fromJsonBytes`. It then **verifies the substitution took**, so a rename of that field upstream fails every merge loudly on the first record rather than drifting into wrong identities. The durable cleanup remains an additive, non-breaking `newTrace(byte[] traceId, …)` overload in `kafka-isotope`, after which the substitution collapses into a single call — but it is a simplification, not a prerequisite, and no change outside this repository was required to ship this.

Three implementation decisions follow from constraints already documented here. First, the edge topic is typed — **Avro+Schema Registry** on CP, **proto-registry** on CCAF, matching each runtime's report sinks — rather than the headers-only representation used by `isotope_consume_edge_markers`. Those markers are written by a Kafka client that has only headers to work with; Flink writes these, so a schema is available, and §3.2 rules out representing multiple contributing traces as repeated same-key headers anyway. Second, the merged record and its edges come from **two INSERT statements** rather than one PTF emitting both row types, because splitting tagged PTF output requires a `CREATE VIEW` over the PTF — a shape the CP Flink 2.1.2 Expander round-trip rejects, as documented in `60_stuck_trace_report.fql`. Third, the capability is **opt-in on both runtimes**, using gating each already had: `count = var.enable_merge_provenance ? 1 : 0` on CCAF, mirroring `var.enable_trace_rca`, and the formerly unused `args` in `IsotopeReportsJob.main` on CP. Disabled — the default — no extra DDL is applied, no extra INSERT joins the statement set, and no extra topic is created.

Two costs should be explicit. The edge stream emits one record per contributing trace per window, a material fraction of that stage's write volume — an inherent cost of materializing a DAG edge list rather than a peculiarity of this design. And a trace walk still **terminates at the merge boundary**. The merge edges make ancestry queryable, not recursively traversable in Flink SQL, which lacks recursive CTEs. Closing over multiple generations of merges therefore belongs outside Flink, in a relational store — see below. In this design, **"full provenance" means that every derivation edge is recorded, not that a single Flink SQL query returns the complete ancestor set.**

> A recursive CTE (Common Table Expression) is a SQL subquery that repeatedly references its own results, allowing hierarchical or graph-like relationships to be traversed until the complete result set is reached.

That blocker is not reachable in this deployment. `80_merge_collector.fql` fixes its input to the origin stream with `WHERE this_topic = 'orders.placed'`, and `orders.flink_batched` is deliberately excluded from the `isotope_raw` union in `00_source_table.fql`. A merge output therefore cannot re-enter a merge: **ancestry depth is exactly 1**, and the single join described in §2.4

already returns the complete parent set. Recursion becomes necessary only if a second merge stage is added that consumes a merge output.

When it does, the exit is Postgres. `isotope_merge_edge_markers` is already an adjacency list — `(merge_trace_id, contributing_trace_id)` is `(child, parent)` — which is precisely the shape `WITH RECURSIVE` wants, so the edge rows sink unchanged:

```
CREATE TABLE isotope_merge_edge (
    merge_trace_id        text   NOT NULL,
    contributing_trace_id text   NOT NULL,
    window_start          bigint NOT NULL,
    window_end            bigint NOT NULL,
    operator              text   NOT NULL,
    pipeline              text   NOT NULL,
    contributing_service  text,
    contributing_topic    text,
    PRIMARY KEY (merge_trace_id, contributing_trace_id)
);
CREATE INDEX ON isotope_merge_edge (contributing_trace_id);

WITH RECURSIVE ancestry AS (
    SELECT merge_trace_id AS root, contributing_trace_id AS ancestor, 1 AS
depth,
           ARRAY[merge_trace_id, contributing_trace_id] AS path
    FROM isotope_merge_edge
    WHERE merge_trace_id = $1
  UNION ALL
    SELECT a.root, e.contributing_trace_id, a.depth + 1,
           a.path || e.contributing_trace_id
    FROM ancestry a
    JOIN isotope_merge_edge e ON e.merge_trace_id = a.ancestor
    WHERE a.depth < 16                                  -- depth cap
      AND NOT e.contributing_trace_id = ANY(a.path)     -- cycle guard
)
SELECT * FROM ancestry;
```

Three things in that schema are load-bearing. The **composite primary key** makes the sink idempotent under Kafka's at-least-once delivery, and it works only because the merge ID is a pure function of `(pipeline, operator, window_start, window_end, group_key)`: a replayed window regenerates byte-identical edge rows, which collide on the key rather than duplicating. The **depth cap and cycle guard** are not decoration — a recursive CTE over a mis-grained edge set does not terminate, and the database has no way to know Flink cannot currently produce a cycle. And the **sink mechanism breaks the runtime symmetry** the rest of this feature maintains: CP can write Postgres from a Flink JDBC sink or Kafka Connect, while CCAF Flink SQL writes only to Confluent-managed Kafka tables, so it would need a fully-managed Postgres Sink connector instead. One topic, two delivery paths, and `ENABLE_MERGE_PROVENANCE` would no longer describe the whole feature. That asymmetry, against a limitation nothing in this pipeline can currently reach, is why this is documented rather than built.

This is now implemented, and **off by default** — see §2.4 for what it deploys and how to enable it. What has not changed is the boundary itself: **in-band propagation remains deliberately restricted to 1:1 transformations**, where a single isotope can truthfully represent the derivation. Fan-in provenance does not relax that rule — it records, out-of-band, exactly the edges the rule forbids an isotope from claiming.

3.2 Header Keys Must Be Unique

Confluent's ALTER TABLE reference states multi-key headers are unsupported. Isotope uses distinct keys throughout, but this rules out ever expressing hops as repeated same-key headers.

3.3 Windowed Fan-in Only

The merged record's identity is *derived*, not minted — `ISOTOPE_MERGE_TRACE(pipeline, operator, window_start_ms, window_end_ms, group_key, …)` — and that derivation needs an input set that is **bounded, deterministic, and nameable**. A tumbling window is all three, which is why the merge collector is scoped to windowed aggregates.

Nothing else in Flink SQL offers that key. An unbounded `GROUP BY`, a regular (non-windowed) join, and a dedup (`ROW_NUMBER() = 1`) all produce a *changelog*: the output row is revised as inputs arrive, and each revision has a different parent set. There is no window bound to hash, and no stable version exposed in SQL to hash instead. Provenance for those is not this mechanism with different arguments — it would have to become a changelog itself, inserting an edge on a contributor's `+I`/`+U` and retracting it on `-U`/`-D`, with the merged record's identity keyed on the group key alone.

The escape hatch, if that case ever matters, is the one this project already uses twice: a `ProcessTableFunction` holds state, so it can mint an identity per key and remember it without a window. Note before starting that CCAF rejects `MapView`/`ListView` in PTF state — a plain `Map`/`List` field is the documented replacement — and still rejects a `byte[]` map *value*, where a Base64 `String` works. Both fail at `CREATE FUNCTION` time, not at runtime.

3.4 Append-only Sources Only

The `isotope` view is a projection with a filter over the `isotope_raw` union, and a projection inherits its input's changelog mode. Everything downstream therefore assumes the sources are **append-only**, which the demo's `'connector' = 'kafka'` tables are.

Point it at `upsert-kafka` or a CDC source and most of this fails loudly rather than silently: window TVF aggregation refuses updating input, so the merge collector and the five windowed reports will not compile. The dangerous exception is the **1:1 collector** (`75_flink_collector.fql`), a plain projection that *will* run — where `ISOTOPE_APPEND_HOP` would stamp a hop on every `-U`/`+U`. A record being revised is not a record taking a hop, so hop counts inflate and the topology reports a movement that never happened: exactly the fabrication §3.1 exists to prevent, arriving through a different door. Adapting to an upsert world means deciding what a hop *means* under revision before touching any SQL.

4.0 What neither addition changes

Neither addition — the always-on 1:1 collector, nor the opt-in fan-in provenance of §2.4 — changes anything below. Each point is argued where it belongs, in §2.4 and §3.1; this is the consolidated blast radius, for deciding what adopting either one costs.

The services. Out-of-band propagation is untouched. `IsotopeContext`, `IsotopeProducerInterceptor`, and the `interceptor.classes` wiring in `App.java` behave exactly as before, and remain the right model for the services: one thread owns a whole consume→produce hop, so the `ThreadLocal` is valid everywhere it runs.

The wire format. No new field, no new header key, no change to `MAX_HOPS` or `truncated`. A merged record carries an ordinary isotope — a fresh trace with a single hop — and the many-to-one edges it cannot express live out-of-band on their own topic, the same way `isotope_consume_edge_markers` already carries consume edges. All three producers — the interceptor, `ISOTOPE_APPEND_HOP`, and `ISOTOPE_MERGE_TRACE` — emit the identical header shape, so `05_isotope_view.fql` cannot tell them apart.

The reports. `isotope_raw` and the typed views are unchanged, and all seven reports — eight on CCAF with `enable_trace_rca` — read exactly what they read before. `orders.flink_enriched` and `orders.flink_batched` are both deliberately kept out of the `isotope_raw` union: the union is typed `MAP<STRING, BYTES>` and the collector sinks are `MAP<STRING, STRING>`. That exclusion is what makes both collectors additive rather than a schema change.

The default deployment. Fan-in provenance is off unless `ENABLE_MERGE_PROVENANCE=true`. Off, neither runtime creates a table, a topic, or a statement for it, and the deployment is byte-identical to a build without the feature. The two UDFs are registered either way — registration is inert until a statement calls one, which is what lets the switch be flipped without re-uploading an artifact.